

DSA

Algorithms — Deep Dive

Sorting, Searching, Graphs, Greedy, and Dynamic Programming Explained

Java Agentic AI — Knowledge Base Reference

1. Complexity Analysis — The Foundation

Big-O notation describes how an algorithm's running time or memory usage grows as input size n grows, ignoring constant factors and lower-order terms — it answers 'how does this scale?' not 'how fast is it on my machine today?'. Big-Omega describes a lower bound (best case), and Big-Theta describes a tight bound (both upper and lower).

Complexity	$n=10$	$n=1,000$	$n=1,000,000$	Typical algorithm
$O(1)$	1	1	1	Hash lookup
$O(\log n)$	~ 3	~ 10	~ 20	Binary search
$O(n)$	10	1,000	1,000,000	Linear scan
$O(n \log n)$	~ 33	$\sim 10,000$	$\sim 20,000,000$	Merge sort
$O(n^2)$	100	1,000,000	10^{12}	Bubble sort

Note: The jump from $O(n \log n)$ to $O(n^2)$ is dramatic at scale — at 1 million elements, a quadratic algorithm doing $\sim 10^{12}$ operations would take hours, while $n \log n$ does ~ 20 million, finishing in milliseconds. This is why sorting algorithm choice matters far more at scale than in small demos.

2. Sorting Algorithms — Deep Dive

2.1 Merge Sort — Divide and Conquer

Merge sort recursively splits the array in half until each sub-array has one element (trivially sorted), then merges sorted halves back together. The merge step compares the front of each half and takes the smaller, which is what guarantees $O(n)$ work per merge level and $O(n \log n)$ overall since there are $\log n$ levels of splitting.

```
void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    System.arraycopy(temp, 0, arr, left, temp.length);
}
```

Property	Value
Time (best/avg/worst)	$O(n \log n)$ in all cases — always splits evenly
Space	$O(n)$ auxiliary array
Stable	Yes
Good for	Linked lists (no random access penalty), external/large-file sorting, guaranteed worst case

2.2 Quick Sort — Partitioning

Quick sort picks a pivot, partitions the array so smaller elements land left of the pivot and larger land right, then recursively sorts each side. Unlike merge sort it sorts in place, but its worst case (already-sorted input with a naive pivot choice) degrades to $O(n^2)$.

```
void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

int partition(int[] arr, int low, int high) {
    int pivot = arr[high];           // Lomuto partition scheme
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr, i, j);
        }
    }
    swap(arr, i + 1, high);
    return i + 1;
}
```

Note: Choosing a random pivot (or median-of-three) instead of always the last element avoids the $O(n^2)$ worst case on already-sorted or adversarially-crafted input, making average-case $O(n \log n)$ behavior far more reliable in practice.

2.3 Heap Sort

Builds a max-heap from the array ($O(n)$), then repeatedly extracts the maximum (swap root with last element, shrink heap, sift down) — giving guaranteed $O(n \log n)$ worst case with $O(1)$ extra space, at the cost of not being stable and having worse cache locality than quicksort in practice.

3. Searching

3.1 Binary Search and Its Variants

```
// Find the leftmost position where target could be inserted (lower_bound)
int lowerBound(int[] arr, int target) {
```

```

int left = 0, right = arr.length;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] < target) left = mid + 1;
    else right = mid;
}
return left;
}

```

Binary search requires sorted data and repeatedly halves the search space — $O(\log n)$. Its variants (first/last occurrence, search in rotated sorted array, search on answer/binary search over a monotonic predicate) appear constantly in interview problems, so understanding the left/right boundary invariant is more valuable than memorizing one exact template.

4. Graph Algorithms

4.1 Representations

Representation	Space	Edge lookup	Best for
Adjacency Matrix	$O(V^2)$	$O(1)$	Dense graphs, frequent edge existence checks
Adjacency List	$O(V + E)$	$O(\text{degree})$	Sparse graphs (most real-world graphs)

4.2 BFS and DFS

BFS explores level by level using a queue — it finds the shortest path in an unweighted graph. DFS explores as deep as possible before backtracking, using recursion or an explicit stack — useful for cycle detection, topological sort, and connected components.

```

void bfs(int start, List<List<Integer>> adj) {
    boolean[] visited = new boolean[adj.size()];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start); visited[start] = true;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}

```

4.3 Dijkstra's Algorithm — Shortest Path (Non-Negative Weights)

```

int[] dijkstra(int src, List<List<int[]>> adj, int n) { // adj: node -> [neighbor, weight]
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // [node, dist]
}

```

```

pq.offer(new int[]{src, 0});

while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int node = cur[0], d = cur[1];
    if (d > dist[node]) continue;           // stale entry, skip
    for (int[] edge : adj.get(node)) {
        int next = edge[0], weight = edge[1];
        if (dist[node] + weight < dist[next]) {
            dist[next] = dist[node] + weight;
            pq.offer(new int[]{next, dist[next]});
        }
    }
}
return dist;
}

```

Note: Dijkstra greedily picks the closest unvisited node each iteration, using a min-heap for $O((V+E) \log V)$ time. It fails with negative edge weights because it never revisits a node once finalized — Bellman-Ford handles negative weights (and detects negative cycles) at $O(V \cdot E)$ cost instead.

4.4 Union-Find (Disjoint Set Union)

Efficiently tracks a partition of elements into disjoint sets, supporting near- $O(1)$ amortized union and find operations via path compression and union by rank — the backbone of Kruskal's MST algorithm and cycle detection in undirected graphs.

```

int[] parent, rank;

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}

void union(int x, int y) {
    int rootX = find(x), rootY = find(y);
    if (rootX == rootY) return;
    if (rank[rootX] < rank[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
    parent[rootY] = rootX;
    if (rank[rootX] == rank[rootY]) rank[rootX]++;
}

```

5. Greedy Algorithms

A greedy algorithm makes the locally optimal choice at each step, hoping it leads to a globally optimal solution. This only works when the problem has the **greedy-choice property** (a locally optimal choice never precludes reaching the global optimum) and **optimal substructure**. Proving greedy correctness (often via an exchange argument) is what distinguishes a rigorous solution from a lucky guess.

```

// Activity selection - maximize number of non-overlapping activities
int maxActivities(int[][] activities) { // each: [start, end]
    Arrays.sort(activities, (a, b) -> a[1] - b[1]); // sort by END time
    int count = 1, lastEnd = activities[0][1];
}

```

```

for (int i = 1; i < activities.length; i++) {
    if (activities[i][0] >= lastEnd) {
        count++;
        lastEnd = activities[i][1];
    }
}
return count;
}

```

Note: *Sorting by end time (not start time or duration) is the key insight — always picking the activity that finishes earliest leaves the maximum remaining time for future activities, which is the exchange-argument proof of correctness for this classic greedy problem.*

6. Dynamic Programming — Deep Dive

DP applies when a problem has **overlapping subproblems** (the same subproblem recurs many times in a naive recursive solution) and **optimal substructure** (the optimal solution can be built from optimal solutions to subproblems). Two implementation styles: **top-down** (recursion + memoization) and **bottom-up** (iterative tabulation).

```

// 0/1 Knapsack - bottom-up
int knapsack(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i - 1][w]; // don't take item i
            if (weights[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]); // take item i
            }
        }
    }
    return dp[n][capacity];
}

// Longest Common Subsequence
int lcs(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++) {
        for (int j = 1; j <= b.length(); j++) {
            dp[i][j] = (a.charAt(i - 1) == b.charAt(j - 1))
                ? dp[i - 1][j - 1] + 1
                : Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[a.length()][b.length()];
}

```

- Recognizing a DP problem: look for phrases like 'maximum/minimum ways,' 'count the number of ways,' or 'longest/shortest sequence satisfying a property,' especially when brute force involves exponential branching with repeated states.

- Define the state precisely (what does $dp[i][j]$ represent?) before writing any code — most DP bugs come from an ambiguous state definition, not the transition logic.
- Space optimization: if $dp[i]$ only depends on $dp[i-1]$, you can often collapse an $O(n*m)$ table to $O(m)$ using two rolling rows.

7. Algorithm Design Paradigms Summary

Paradigm	Idea	Example
Divide and Conquer	Split into independent subproblems, solve, combine	Merge sort, quick sort, binary search
Greedy	Locally optimal choice at each step	Activity selection, Huffman coding, Dijkstra
Dynamic Programming	Cache overlapping subproblem results	Knapsack, LCS, edit distance
Backtracking	Try, recurse, undo if it fails	N-Queens, subset/permutation generation, Sudoku

8. Quick Interview Recap

- When would you pick merge sort over quicksort? When you need a guaranteed $O(n \log n)$ worst case and stability (e.g. sorting records by a secondary key), or when sorting a linked list where random access is expensive.
- Why does Dijkstra fail with negative weights? It finalizes a node's shortest distance once popped from the priority queue and never revisits it, but a negative edge encountered later could still improve that distance — Bellman-Ford handles this by relaxing all edges $V-1$ times.
- What are the two conditions required for dynamic programming to apply? Overlapping subproblems and optimal substructure.
- Greedy vs DP — how do you tell which one a problem needs? If making the locally best choice at each step can be proven to never lose future optimality (often via an exchange argument), greedy suffices and is simpler/faster; if a locally optimal choice can foreclose a better global solution, you need DP to explore/cache all viable subproblem states.